

In-Sample Modeling and 1% Value at Risk for Percentage Log Stock Returns Using Maximum Likelihood

February 12, 2026

Abstract

We study daily percentage log-returns of 25 DJIA stocks and fit several parametric models by maximum likelihood: a Gaussian distribution, mixtures of two and three Gaussians, a location-scale non-central t distribution, and a shifted weighted sum of independent χ_1^2 variables. For each stock we report the maximized log-likelihood and the corresponding AIC and BIC. We also compute the 1% Value-at-Risk (VaR) from the empirical sample quantile and from each fitted model, and compare the results across models and stocks.

1 Introduction

The goal of this assignment is to investigate how different parametric models capture the empirical distribution of daily stock returns and how these modeling choices affect the estimation of risk measures such as Value-at-Risk (VaR). We work with daily percentage log-returns of the 25 DJIA stocks over a long historical sample.

For each stock, we fit several alternative distributions by maximum likelihood: a single Gaussian model, two- and three-component Gaussian mixtures, a weighted sum of independent χ_1^2 variables plus a location term, and a location-scale non-central t model. Model fit is assessed using log-likelihood, AIC and BIC. Finally, we compute the 1% VaR both empirically and under each fitted model, and compare the results stock by stock.

All numerical work is done in MATLAB R2024b. In the next sections we describe the models, the estimation procedure, and how we compare the fitted models and compute Value-at-Risk.

1.1 Data and preprocessing

We use daily returns data for 25 DJIA constituents, working with percentage log-returns. For numerical stability in some routines that use numerical integration, we remove a small number of extreme observations (three smallest and three largest). When we do this, the same trimmed sample is used across models for that stock.

2 Models and density functions

In this section we describe the four models and how we implement their probability density functions (pdfs) in MATLAB. These functions are used later in the likelihood and estimation.

2.1 Gaussian distribution

In the first model we assume that a return X follows a Gaussian distribution with mean μ and variance σ^2 ,

$$X \sim N(\mu, \sigma^2).$$

The pdf of X is

$$f_{\text{Gauss}}(x; \mu, \sigma) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{1}{2} \left(\frac{x - \mu}{\sigma}\right)^2\right).$$

We implement this density in MATLAB in the function `pdf_gauss`. The function takes a vector (or matrix) of evaluation points `x` and returns the corresponding pdf values. It checks that `sigma` is positive and then vectorizes the formula above.

```

1 function f = pdf_gauss(x, mu, sigma)
2     if sigma <= 0
3         error('sigma must be positive. ');
4     end
5     constant_term = 1 ./ (sigma .* sqrt(2 * pi));
6     exp_term = exp(-0.5 .* ((x - mu) ./ sigma).^2);
7     f = constant_term .* exp_term;
8 end

```

Listing 1: Gaussian pdf

2.2 Mixture of Gaussians

In the second model we use a mixture of Gaussian distributions:

$$f(x) = \sum_{i=1}^k \lambda_i f_{\text{Gauss}}(x; \mu_i, \sigma_i),$$

where μ_i are component means, $\sigma_i > 0$ are component standard deviations, and the weights satisfy $\lambda_i > 0$ and $\sum_{i=1}^k \lambda_i = 1$. In this project we use $k = 2$ and $k = 3$.

In MATLAB we evaluate this density with `pdf_mog`. The function reshapes x to a column vector and μ, σ, λ to row vectors, computes all component Gaussian pdf values using `pdf_gauss`, and returns their weighted sum using λ (note that the sum of elements of λ is 1).

The MATLAB code is:

```

1 function f = pdf_mog(x, mu, sigma, lambda)
2     x = x(:);
3     mu = mu(:)';
4     sigma = sigma(:)';
5     lambda = lambda(:)';
6     k = length(mu);
7
8     component_pdfs = pdf_gauss(x, mu, sigma);
9     f = component_pdfs * lambda';
10 end

```

Listing 2: Mixture of Gaussians pdf: pdf_{mog}

2.3 Weighted sum of χ^2 variables

Let $C_j \sim \chi_1^2$ be independent for $j = 1, \dots, K$. We model

$$X = \mu + \sum_{j=1}^K a_j C_j,$$

where μ is a location parameter and $a = (a_1, \dots, a_K)$ are the weights.

We compute the pdf by inverting the characteristic function. For $C_j \sim \chi_1^2$,

$$\varphi_{C_j}(t) = (1 - 2it)^{-1/2}.$$

By independence, for the sum $Y = \sum_{j=1}^K a_j C_j$ without the shift we have

$$\varphi_Y(t) = \prod_{j=1}^K (1 - 2ia_j t)^{-1/2}.$$

Since $X = \mu + Y$, the characteristic function of X is

$$\varphi_X(t) = e^{i\mu t} \varphi_Y(t).$$

In the code we handle the shift by evaluating the density at $x - \mu$, i.e. we compute $f_Y(x - \mu)$.

In MATLAB we use the following function for the product term $\varphi_Y(t)$:

```
1 function cf = cf_wghd_sum(t, a)
2     denom = (1 - 2i*a*t);
3     cf = prod( denom.^(-1/2) );
4 end
```

Listing 3: Characteristic function product term

The inversion formula we use for Y is

$$f_Y(y) = \frac{1}{\pi} \int_0^\infty \operatorname{Re} (e^{-ity} \varphi_Y(t)) dt.$$

Therefore, for $X = \mu + Y$ we use

$$f_X(x) = f_Y(x - \mu).$$

In the code we truncate the integral at $T = 2000$ and evaluate it with MATLAB's `integral`.

```
1 function f = pdf_wghd_sum(x, k, a)
2     T = 2000;
3     integrand = @(t) real(exp(-1i*t*x)) * cf_wghd_sum(t, k, a));
4     f = 1/(pi) * integral(integrand, 0, T, ...
5         'Arrayvalued', true);
6 end
```

2.4 Location-scale non-central t distribution

In the fourth model we use a location-scale non-central t distribution. Let T follow a non-central t distribution with degrees of freedom $\nu > 0$ and non-centrality parameter δ ,

$$T \sim \text{NCT}(\nu, \delta).$$

We then define

$$Y = \mu + \sigma T,$$

where μ is a location parameter and $\sigma > 0$ is a scale parameter. The pdf of Y is obtained by a standard change of variables,

$$f_Y(y; \mu, \sigma, \nu, \delta) = \frac{1}{\sigma} f_T\left(\frac{y - \mu}{\sigma}; \nu, \delta\right),$$

where f_T is the standard non-central t density.

MATLAB has a built-in function `nctpdf` that evaluates $f_T(z; \nu, \delta)$ for a vector of z values. We use this function inside our `pdf_nct`. We standardize x to $z = \frac{x - \mu}{\sigma}$. In the end we call `nctpdf` and multiply by the factor $\frac{1}{\sigma}$ from the change of variables.

The MATLAB code is:

```

1 function f = pdf_nct(x, mu, sigma, nu, delta)
2     z = (x - mu) ./ sigma;
3     pdf_standard = nctpdf(z, nu, delta);
4     f = (1 ./ sigma) .* pdf_standard;
5     f = f(:);
6 end

```

Listing 5: Location-scale NCT pdf: `pdfnct`

These four functions `pdf_gauss`, `pdf_mog`, `pdf_weighted_chisq`, and `pdf_nct` provide exact (up to numerical integration error) pdf evaluations for all models. In the next step of the project we use these functions to compute the likelihoods and to estimate the model parameters by maximum likelihood for each stock.

3 Maximum Likelihood Estimation

In this part we estimate the parameters of each model using maximum likelihood. We treat the time series as IID observations, as asked in the assignment. For one stock we observe an IID

sample

$$x_1, \dots, x_T.$$

Each model has a pdf $f(x; \theta)$ with parameter vector θ . The likelihood is

$$L(\theta) = \prod_{t=1}^T f(x_t; \theta).$$

We do not maximize $L(\theta)$ directly. Instead we minimize the negative log-likelihood (NLL)

$$\text{NLL}(\theta) = - \sum_{t=1}^T \log f(x_t; \theta).$$

This is the objective function used in all optimization scripts.

We minimize the NLL numerically using MATLAB. For the Gaussian model, non-central t and the mixture models we use `fminsearch`. For the weighted χ^2 model we use MATLAB's `mle` with a custom negative log-likelihood function passed through the `'nloglf'` argument. In both cases the optimizer works with an unconstrained parameter vector. Therefore, in each model-specific NLL we include basic parameter checks that are needed for the density evaluation (for example $\sigma > 0$ in the Gaussian case, and valid mixture weights in the mixture case). When a parameter vector is not valid for that model, the corresponding NLL returns `Inf` (or a large penalty value, depending on the model implementation). For numerical stability, before taking logarithms we replace very small density values by a lower bound (such as `realmin`).

3.1 Gaussian MLE

For the Gaussian model we estimate μ and σ with the constraint $\sigma > 0$. We minimize the negative log-likelihood

$$\text{NLL}(\mu, \sigma) = - \sum_{t=1}^T \log f_{\text{Gauss}}(x_t; \mu, \sigma)$$

using `fminsearch`. We start at the sample mean and sample standard deviation. Since `fminsearch` is unconstrained, the NLL returns `Inf` if $\sigma \leq 0$. For numerical stability we replace density values smaller than `realmin` by `realmin` before taking logs. After optimization we take `abs` of the fitted σ to ensure a positive output.

```
1 function [params, nll] = fit_gaussian(X, initvec)
2 opts = optimset('Display','off','MaxIter',500, 'TolX', 1e-6);
3 nll_fun = @(theta) gaussian_nll(theta, X);
4 [params, nll] = fminsearch(nll_fun, initvec, opts);
5 params(2) = abs(params(2));
6 end
7
8 function nll = gaussian_nll(theta, X)
9 mu = theta(1); sigma = theta(2);
```

```

10 if sigma <= 0, nll = Inf; return; end
11 f = pdf_gauss(X, mu, sigma);
12 f(f <= realmin) = realmin;
13 nll = -sum(log(f));
14 end

```

Listing 6: Gaussian MLE (fit + NLL)

3.2 Mixture of Gaussians MLE

We fit mixtures with $k = 2$ and $k = 3$ components. The parameters are the component means μ_i , standard deviations σ_i , and weights λ_i . The constraints are $\sigma_i > 0$, $\lambda_i \in (0, 1)$, and $\sum_{i=1}^k \lambda_i = 1$.

We use `fminsearch` to minimize the negative log-likelihood. Since `fminsearch` is unconstrained, we enforce the weight-sum constraint by optimizing only the first $k - 1$ weights and setting the last one as

$$\lambda_k = 1 - \sum_{i=1}^{k-1} \lambda_i.$$

If the resulting weights are not all in $(0, 1)$, or if any σ_i is too small, we return `Inf` in the NLL to reject that parameter vector.

```

1 function [params, nll] = fit_mog(X, init_mu, init_sigma,
   init_lambda)
2 k = length(init_mu);
3 initvec = [init_mu(:); init_sigma(:); init_lambda(1:k-1)];
4 opts = optimset('Display','off','MaxIter',1000, 'TolX', 1e-5);
5 [estim_vec, nll] = fminsearch(@(p) mog_nll(p, X, k), initvec, opts
   );
6 params.mu = estim_vec(1:k)';
7 params.sigma = abs(estim_vec(k+1:2*k)');
8 lam_partial = estim_vec(2*k+1:end);
9 params.lambda = [lam_partial; 1 - sum(lam_partial)]';
10 end

```

Listing 7: MoG: parameter vector and optimization

Inside the NLL we unpack the parameter vector, compute the last weight, check the constraints, evaluate the mixture density, and sum the log-densities. Before taking logs we clip density values at `realmin` to avoid $\log(0)$.

```

1 function nll = mog_nll(theta, X, k)
2 mu = theta(1:k);
3 sigma = theta(k+1:2*k);
4 lam_partial = theta(2*k+1:end);
5 last_lam = 1 - sum(lam_partial);

```

```

6 lambda = [lam_partial; last_lam];
7
8 if any(sigma <= 1e-6) || any(lambda <= 0) || any(lambda >= 1)
9     nll = Inf;
10     return;
11 end
12
13 f = pdf_mog(X, mu', sigma', lambda');
14 f(f <= realmin) = realmin;
15 nll = -sum(log(f));
16 end

```

Listing 8: MoG: negative log-likelihood with constraints

3.3 Weighted sum of IID χ^2 variables MLE

We fit the IID model

$$X = \mu + \sum_{i=1}^K a_i C_i, \quad C_i \sim \chi_1^2$$

by maximum likelihood. The parameter vector is

$$\theta = (\mu, a_1, \dots, a_K).$$

We use MATLAB's `mle()` with a custom negative log-likelihood. For a given weight vector, we shift the data by μ , and evaluate the pdf with `pdf_wghtd_sum`. The objective is

$$\text{NLL}(\theta) = - \sum_{t=1}^T \log \left(f(x_t - \mu; a_1, \dots, a_K) \right).$$

We use `'omitnan'` because some trial parameter values during optimization can produce NaN from the numerical integration.

First we define the custom NLL.

```

1 function nll = nloglf_wchi(params, data, K)
2     mu = params(1);
3     a  = params(2:end);
4     nll = -sum(log( ...
5         pdf_wghtd_sum((data - mu), K, a) ...
6     ), 'omitnan');
7 end

```

Listing 9: Custom NLL for the weighted χ^2 model

Then we wrap the estimation for one stock returns series, which we pass as `ret_percent`. The wrapper converts percentage returns to log-returns, sets the `mle` options, and calls `mle`. The

parameters for estimation are

$$[\mu, a_1, \dots, a_K].$$

Before calling `mle` we must choose a starting vector for the parameters. For this model we did not use a single optimization rule. Instead, the starting vector (initial guess for the parameter values) was adjusted accordingly when fitting the different stock returns. This was needed to help the optimizer converge. Due to issues with the optimizer's convergence, the fitting of these models took the most significant amount of time among all.

```
1 function pars = fit_wchi_mle_one(ret_percent, K, start, opts)
2     data = log(1 + ret_percent(:)/100);
3
4     if nargin < 3
5         start = [];
6     end
7     if nargin < 4 || isempty(opts)
8         opts = statset('MaxIter',1500, 'MaxFunEvals',100000, ...
9                       'Display','iter','TolX',1e-7,'TolFun',1e-7)
10        ;
11    end
12    pars = mle(data, ...
13              'nloglf', @(params,data,cens,freq) nloglf_wchi(params,
14                    data, K), ...
15              'Start', start, ...
16              'Options', opts);
17 end
```

Listing 10: Wrapper: MLE for one stock

3.4 Non-Central t MLE

We estimate the location μ , scale σ , degrees of freedom ν , and non-centrality parameter δ . We constrain σ and ν to be positive, minimize the negative log-likelihood using `fminsearch`, rejecting invalid parameter values by returning a large penalty, and we clip the p.d.f. values to avoid $\log(0)$.

```
1 function [params, nll] = fit_nct(X, initvec)
2 opts = optimset('Display','off','MaxIter',500, 'TolX', 1e-5);
3 [params, nll] = fminsearch(@(p) nct_nll(p, X), initvec, opts);
4 end
5
6 function nll = nct_nll(theta, X)
7 mu = theta(1); sigma = theta(2);
```

```

8 nu = theta(3); delta = theta(4);
9 if sigma <= 0 || nu <= 0.1 || nu > 100
10     nll = 1e10; return;
11 end
12 f = pdf_nct(X, mu, sigma, nu, delta);
13 f(f <= 1e-20) = 1e-20;
14 nll = -sum(log(f));
15 end

```

Listing 11: NCT MLE optimization

4 In-sample fit for Stock 1 (KDE and fitted densities)

In this part we visualize the fit of all models on the percentage log returns of the first stock. We plot a histogram of the data and a kernel density estimate (KDE) as a nonparametric reference. Then we overlay the fitted model densities using the maximum likelihood estimates from Section 3: Gaussian, Mixture of Gaussians (MoG) with $k = 2$, MoG with $k = 3$, weighted sum of K χ^2 r.v.s model, $K = 2, 3, 4, 5$, and the non-central t model.

The KDE is computed in MATLAB using `ksdensity`. All parametric curves are evaluated on the same grid using the pdf functions defined in Section 2: `pdf_gauss`, `pdf_mog`, `pdf_wghtd_sum`, and `pdf_nct`. We plot the fitted distributions for Stock 1.

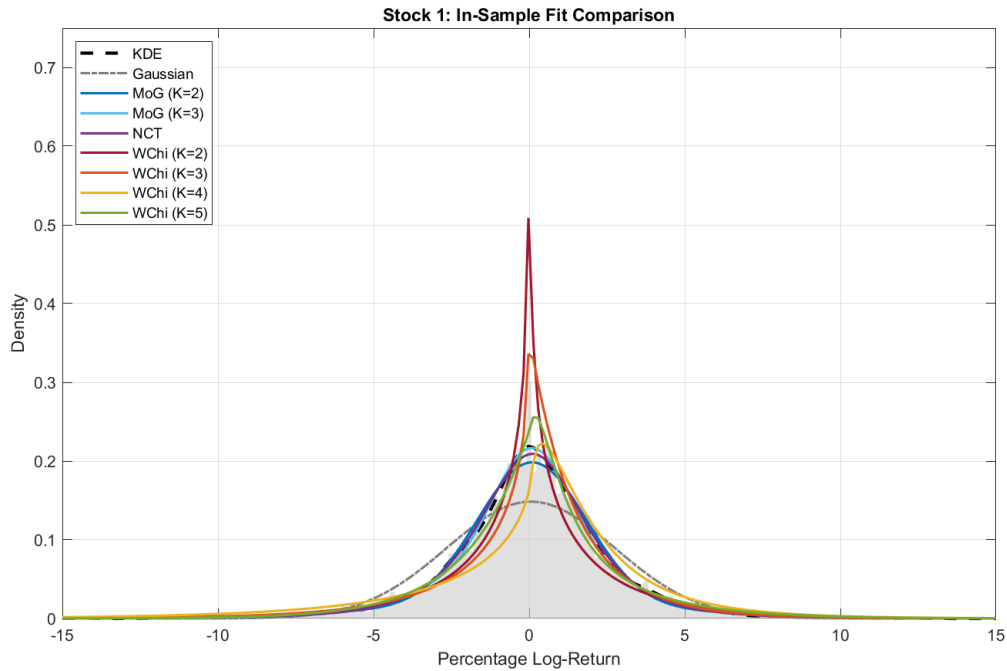


Figure 1: Stock 1: KDE and fitted model densities.

Figure 1 compares the KDE with the fitted parametric densities for Stock 1.

The Gaussian density is symmetric and has thin tails. In the plot it does not match the KDE well in the tails and it also does not fit well around the center of the data. This is consistent with daily stock returns typically having heavier tails than those of a Gaussian model.

MoG2 and MoG3 are closer to the KDE compared to the Gaussian in the center and in the tails. A mixture model can create heavier tails and higher p.d.f. values around the center, because it combines components with different variances (and possibly different means). MoG3 is more flexible than MoG2 because it has one additional component.

The NCT model also follows the KDE more closely than the Gaussian, and falls between the MoG2 and MoG3. It can better capture the heavier tails through the degrees of freedom parameter and the asymmetry through the non-centrality parameter.

The weighted χ^2 fits can produce a very sharp peak for small values of K (the number of terms in the sum). In the plot this can show up as a strong and non-smooth spike. When K increases, the fitted curve tends to become smoother, but the overall fit is highly sensitive to the choice of initial parameter values (and this gave rise to many problems with optimization convergence), since they strongly influence the region in which the fitted parameters will end up.

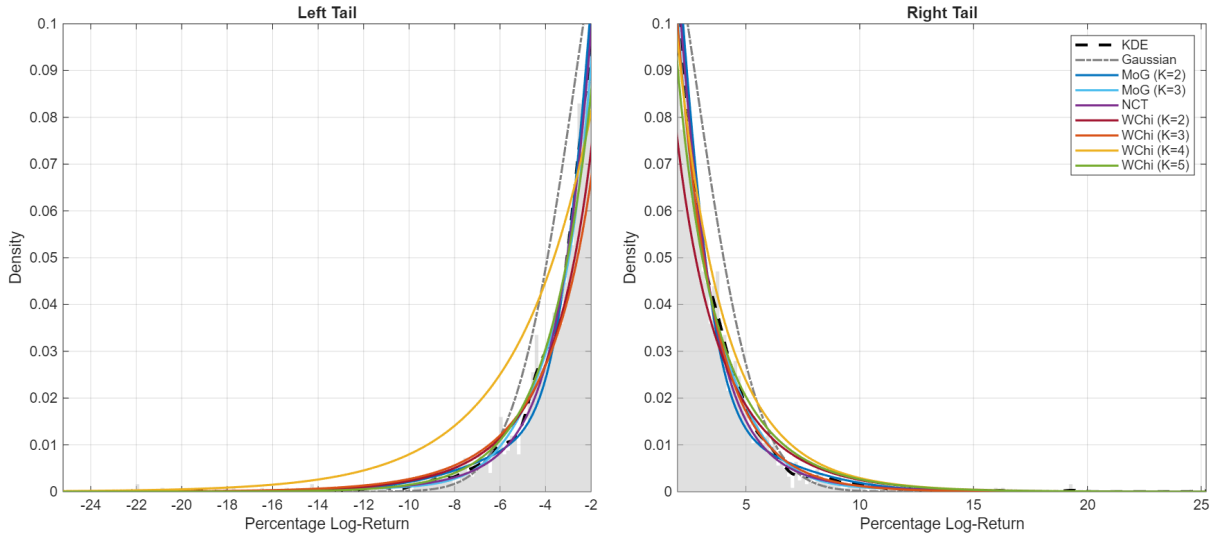


Figure 2: Stock 1: fitted model densities in the left and right tails.

Figure 2 zooms into the left and right tails.

Left tail: the Gaussian is below the KDE over the shown negative-return range. MoG2, MoG3, and NCT are closer to the KDE. The weighted χ^2 curves differ substantially across K ; in this plot $K = 4$ assigns noticeably more mass far into the left tail than the KDE and the other models.

Right tail: the Gaussian drops faster than the KDE in the shown positive-return range. MoG2, MoG3, and NCT stay closer to the KDE over most of the plotted right tail. The weighted χ^2 curves are generally above the KDE in parts of the right tail in this plot, meaning they can assign more mass to large positive returns than the KDE over the plotted range.

5 Model comparison across all assets (log-likelihood, AIC, BIC)

For each of the 25 stocks we fit each candidate model by maximum likelihood and then compute the maximized log-likelihood, Akaike information criterion (AIC), and Bayesian information criterion (BIC).

5.1 Log-likelihood

For a fitted model with parameter estimate $\hat{\theta}$, the in-sample log-likelihood is

$$\ell(\hat{\theta}) = \sum_{t=1}^T \log f(x_t; \hat{\theta}).$$

A higher value means a better in-sample fit.

5.2 AIC

AIC adds a penalty for the number of parameters:

$$\text{AIC} = 2p - 2\ell(\hat{\theta}),$$

where p is the number of free parameters in the model. The penalty is proportional to p and does not depend on T . Models with lower AIC values are preferred.

5.3 BIC

BIC uses a stronger penalty that increases with sample size:

$$\text{BIC} = p \log(T) - 2\ell(\hat{\theta}).$$

Similar to AIC, lower BIC values are preferred when choosing a model.

Remark on LL, AIC, and BIC. Log-likelihood measures in-sample fit only, so models with more parameters often achieve a higher log-likelihood. AIC and BIC include penalties for the number of parameters. Because of this, a model can achieve a relatively high log-likelihood but have high AIC or BIC if the improvement in fit is small relative to the added complexity.

5.4 Best model per stock

For each stock we select the model with the highest in-sample log-likelihood, the model with the lowest AIC, and the model with the lowest BIC. We report them in the table below.

Stock	Best by LL	Best by AIC	Best by BIC
1	MoG3	MoG3	NCT
2	MoG3	MoG3	NCT
3	NCT	NCT	NCT
4	MoG3	MoG3	NCT
5	NCT	NCT	NCT
6	NCT	NCT	NCT
7	MoG3	MoG3	NCT
8	NCT	NCT	NCT
9	NCT	NCT	NCT
10	MoG3	MoG3	WChi3
11	MoG3	MoG3	NCT
12	MoG3	MoG3	NCT
13	NCT	NCT	NCT
14	NCT	NCT	NCT
15	NCT	NCT	NCT
16	MoG3	MoG3	NCT
17	NCT	NCT	NCT
18	MoG3	MoG3	NCT
19	MoG3	MoG3	NCT
20	NCT	NCT	NCT
21	NCT	NCT	NCT
22	NCT	NCT	NCT
23	NCT	NCT	NCT
24	MoG3	MoG3	NCT
25	NCT	NCT	NCT

Table 1: Best in-sample model per stock by log-likelihood (max), AIC (min), and BIC (min).

Table 1 summarizes which model is selected by the three criteria. Two patterns stand out.

First, the model that most often wins by BIC is the non-central t (NCT). This means that, after penalizing for the number of parameters, NCT gives the best overall in-sample fit for most stocks.

Second, the log-likelihood and AIC winner is usually NCT, followed closely by MoG3. The result for log-likelihood is expected, because log-likelihood only measures in-sample fit and does not penalize complexity. In our case the NCT model has 4 parameters and MoG3 has 8 free parameters.

Finally, the penalization of extra parameters in AIC ($2p$) is not enough to offset the high log-likelihood value obtained by MoG3, and so in many cases that model is optimal over NCT.

The weighted χ^2 model is selected only once by AIC/BIC in our results. This model can generate strong skewness and a very sharp peak near the central mass of the data. In our density plots for stock 1, this appears as a spike near zero for some K values. For lower K , the model fits well around the central mass of the data, capturing well its peak, however it is thinner in the tails. For larger K , the model seems to capture the tails better, but fails to fit well to the data in its central parts.

5.5 Limitations

All results are in-sample. This means the comparison is based on how well each model fits the same data it was estimated on. A more complete comparison would also check out-of-sample

performance, but this is not done here. For the weighted χ^2 model, the likelihood and CDF evaluations rely on numerical integration, so some runs can be sensitive to integration settings and starting values. Finally, the data is treated as IID, which we know cannot be correct, for example due to stylized facts of stock returns. Usually the returns exhibit volatility clustering, meaning that high volatility tends to be followed by high volatility, and vice versa, and that behavior is ignored by an IID model.

6 1% Value-at-Risk (VaR)

For each stock returns series we compute the 1% VaR ($\alpha = 0.01$) for the empirical sample and for each fitted model. We use the definition

$$q_\alpha = F^{-1}(\alpha), \quad \text{VaR}_\alpha = -q_\alpha,$$

where F is the c.d.f. of a given model, so that VaR is reported as a positive loss number. The empirical sample VaR is the sample quantile:

$$q_\alpha^{\text{emp}} = \text{quantile}(x, \alpha), \quad \text{VaR}_\alpha^{\text{emp}} = -q_\alpha^{\text{emp}}.$$

```

1 function [q, VaRloss] = var_empirical(x, alpha)
2     q = quantile(x, alpha);
3     VaRloss = -q;
4 end

```

Listing 12: Empirical VaR

For models where MATLAB provides an inverse CDF, we use the built-in function. For models without a closed-form inverse, we compute the CDF and then find q_α by solving $F(q_\alpha) = \alpha$ numerically. In our code this is done by evaluating $F(x)$ on a fine grid, taking the first interval where $F(x) - \alpha$ changes sign, and solving on that interval with `fzero`.

For the Gaussian model we use `norminv`:

```

1 function [q, VaRloss] = var_gauss(mu, sigma, alpha)
2     q = mu + sigma * norminv(alpha);
3     VaRloss = -q;
4 end

```

Listing 13: Gaussian VaR

For the non-central t model we use `nctinv`:

```

1 function [q, VaRloss] = var_nct(mu, sigma, nu, delta, alpha)
2     q = mu + sigma * nctinv(alpha, nu, delta);
3     VaRloss = -q;
4 end

```

Listing 14: Non-central t VaR

For the Gaussian mixtures ($k = 2, 3$) we use the mixture CDF and then solve with `fzero`:

```

1 function [q, VaRloss] = var_mog_grid(mu, sigma, lambda, alpha,
   x_min, x_max)
2     nGrid = 50000;
3     mu = mu(:); sigma = sigma(:); lambda = lambda(:);
4
5     Fx = @(x) local_mog_cdf(x, mu, sigma, lambda);
6
7     xg = linspace(x_min, x_max, nGrid);
8     gg = Fx(xg) - alpha;
9
10    idx = find(gg(1:end-1) <= 0 & gg(2:end) >= 0, 1, 'first');
11    lo = xg(idx); hi = xg(idx+1);
12
13    q = fzero(@(x) Fx(x) - alpha, [lo hi]);
14    VaRloss = -q;
15
16    function F = local_mog_cdf(x, mu, sigma, lambda)
17        x = x(:).';
18        Z = (x - mu) ./ sigma;
19        F = sum(lambda .* normcdf(Z), 1);
20        F = F(:);
21    end
22 end

```

Listing 15: MoG VaR

For the weighted χ^2 model we compute $F(x)$ with the characteristic function using the Gil-Peleaz integral

$$F_X(x) = \frac{1}{2} - \frac{1}{\pi} \int_0^\infty \frac{\text{Im}(e^{-itx} \varphi_X(t))}{t} dt.$$

We make the change of variables $t = \frac{1}{u} - 1$ and integrate over $u \in (1/(T+1), 1)$:

$$F(x) = \frac{1}{2} - \frac{1}{\pi} \int_{1/(T+1)}^1 \frac{\text{Im}\left(e^{-ix\frac{1-u}{u}} \varphi_X\left(\frac{1-u}{u}\right)\right)}{u(1-u)} du,$$

where $\varphi_X(t) = e^{i\mu t} \prod_{i=1}^K (1 - 2ia_i t)^{-1/2}$ is the characteristic function of $X = \mu + \sum_{j=1}^K a_j X_j$, where $X_j \sim \chi_1^2$, $1 \leq j \leq K$, are i.i.d. chi-squared with 1 degree of freedom random variables. After computing $F(x)$, we find q_α by solving $F(q_\alpha) = \alpha$ with the same procedure as before.

```

1 function [q, VaRloss] = var_wchi_grid(mu, a, alpha, x_min, x_max)

```

```

2
3     nGrid = 5000;
4     T = 200;
5
6     a = a(:).';
7     K = numel(a);
8
9     Fx = @(x) local_cdf(x, mu, a, K, T);
10
11     xg = linspace(x_min, x_max, nGrid);
12     Fg = Fx(xg);
13     gg = Fg - alpha;
14
15     idx = find(gg(1:end-1) <= 0 & gg(2:end) >= 0, 1, 'first');
16     lo = xg(idx); hi = xg(idx+1);
17
18     q = fzero(@(x) Fx(x) - alpha, [lo hi]);
19     VarLoss = -q;
20
21     function F = local_cdf(x, mu, a, K, T)
22         x = x(:);
23
24         umin = 1/(T+1);
25         umax = 1 - 1e-10;
26
27         integrand = @(u) local_u_integrand(u, x, mu, a, K);
28
29         I = integral(integrand, umin, umax, ...
30                     'ArrayValued', true, 'RelTol', 1e-4, 'AbsTol'
31                     , 1e-6);
32
33         F = 0.5 - (1/pi) * I;
34         F = max(min(F,1),0);
35     end
36
37     function Y = local_u_integrand(u, x, mu, a, K)
38         u = u(:).';
39         t = (1 - u) ./ u;
40         phi = exp(1i*mu.*t) .* cf_wghstd_sum(t, K, a);
41         Z = exp(-1i*(x * t)) .* (ones(numel(x),1) * phi);
42         Y = (imag(Z) ./ (ones(numel(x),1) * t)) .* (ones(numel(x)
43             ,1) * (u.^(-2)));

```


42 **end**
43 **end**

Listing 16: Weighted χ^2 VaR by CF-CDF inversion

Below we show computed empirical sample VaR and the model-based VaRs for each stock, and we mark the model whose VaR is closest to the empirical sample VaR in absolute difference.

Model	1	2	3	4	5	6	7	8	9
Empirical	7.4557	6.0534	5.8268	5.7013	4.1741	5.0216	5.4563	5.0064	6.8469
Gaussian	6.2011	5.1599	4.9075	4.6948	3.8363	4.3512	4.4929	4.0509	5.6109
MoG2	8.0812	7.0078	6.8519	5.9911	4.9250	5.6790	5.8526	5.4312	7.4732
MoG3	7.2661	6.2089	5.6479	5.6460	4.8321	5.0695	5.3605	4.9543	7.7974
NCT	7.5714	6.6505	5.7325	5.5546	4.3662	5.1767	5.4825	4.9886	6.7243
WChi K=2	8.6185	6.9105	6.5455	6.7477	5.3608	6.0287	6.2178	5.5867	7.8747
WChi K=3	9.2411	7.2951	7.0892	6.3438	4.9808	6.4326	6.6767	5.3162	7.6226
WChi K=4	9.6260	5.5353	7.6715	5.2673	7.1088	7.0218	6.0661	7.2829	9.1746
WChi K=5	7.5436	8.2083	8.8060	5.7479	7.9149	7.4629	7.0610	7.7429	7.5578

Table 2: 1% VaR (loss, positive) for stocks 1–9. Bold indicates the parametric model closest to the empirical VaR.

Model	10	11	12	13	14	15	16	17	18
Empirical	3.6856	6.4514	3.8829	4.1134	4.2950	4.6601	5.5598	5.4986	4.6379
Gaussian	3.1185	5.4896	3.2582	3.5224	3.4038	3.9598	4.6052	4.7221	3.8841
MoG2	3.9381	7.5134	4.2586	4.5214	4.5533	5.3441	5.8870	6.1479	4.7818
MoG3	3.6925	6.5321	3.9051	4.0686	4.6804	4.8121	5.4349	6.3793	4.6057
NCT	3.7657	6.9506	3.9663	4.1736	4.2317	4.5571	5.4942	5.7015	4.5494
WChi K=2	4.3385	7.2156	4.5389	4.9212	4.7203	5.4589	6.3925	6.4318	5.5916
WChi K=3	4.1729	7.5905	4.2598	5.2628	5.0467	5.0533	6.0423	5.9841	5.2110
WChi K=4	4.0779	7.0253	3.4961	4.3419	4.2489	6.1212	6.8271	4.9971	4.7961
WChi K=5	3.6823	7.6998	4.0793	4.8369	7.2444	4.7082	7.3041	5.4844	4.8825

Table 3: 1% VaR (loss, positive) for stocks 10–18. Bold indicates the parametric model closest to the empirical VaR.

Model	19	20	21	22	23	24	25
Empirical	3.6716	4.6956	4.8520	4.2865	4.9225	4.4141	4.3055
Gaussian	3.1294	4.0481	4.0492	3.5332	4.1968	3.7435	3.5857
MoG2	4.0622	5.5378	5.6704	4.4621	5.3350	4.7639	4.7067
MoG3	3.7334	4.6816	5.2871	4.6109	5.5046	4.4885	4.7862
NCT	3.7511	4.7962	4.9812	4.2675	4.8450	4.5445	4.2039
WChi K=2	4.3225	5.6557	5.2779	5.0397	5.9241	5.2294	5.0532
WChi K=3	4.1315	5.2855	5.6542	4.8692	5.6233	5.5632	4.8998
WChi K=4	9.1584	9.5039	6.0816	8.9273	5.9031	5.3603	9.3007
WChi K=5	4.2083	4.7516	6.2139	5.2089	6.3896	5.6098	5.5641

Table 4: 1% VaR (loss, positive) for stocks 19–25. Bold indicates the parametric model closest to the empirical VaR.

Across most stocks, the Gaussian VaR is smaller than the empirical VaR. This is because the fitted normal distribution has thin tails, so the left 1% quantile is less extreme than in the data. Therefore the Gaussian model underestimates left-tail risk.

For MoG2/MoG3 and NCT, the VaR values are usually closer to the empirical VaR. These models allow heavier tails: MoG through a high-variance component and NCT through a low degrees-of-freedom parameter. In the tables this appears as MoG/NCT VaR values are typically larger than the Gaussian VaR and often near the empirical sample VaR.

For the weighted χ^2 models, the VaR is highly sensitive to the estimated weights $a_1, \dots, a_K$. Increasing K increases flexibility, and the fitted model can assign more probability mass to the left tail, which can increase the 1% VaR. The WChi VaR is close to the empirical sample VaR, when the fitted left tail is similar to the sample left tail. In most other cases, the WChi VaR is higher than the empirical sample VaR, and so the fitted WChi left tail is heavier than the sample left tail of the data, but there is a small number of exceptions, where the opposite holds.

A Appendix: Main script with all the MATLAB code

```

1 % 1. Load Data
2
3 data = load('DJIA30stockreturns.mat');
4 raw_returns = data.DJIARet;
5 x_pct = 100 * log(1 + raw_returns / 100);
6 [T_full, nStocks] = size(x_pct);
7
8 % 2. Maximum Likelihood Estimation (Gauss, MoG, NCT)
9
10 % Gauss, Col 2: MoG2, Col 3: MoG3, Col 4: NCT
11 params_other = cell(nStocks, 4);
12 LL_results = NaN(nStocks, 4);

```

```

13 AIC_results = NaN(nStocks, 4);
14 BIC_results = NaN(nStocks, 4);
15
16 for j = 1:nStocks
17     x = x_pct(:, j);
18     x = x(isfinite(x));
19     n = numel(x);
20
21     % --- Gaussian ---
22     mu0 = mean(x); sig0 = std(x);
23     [p_gauss, nll_gauss] = fit_gaussian(x, [mu0, sig0]);
24     params_other{j, 1} = p_gauss;
25     LL_results(j, 1) = -nll_gauss;
26     k_gauss = 2;
27     AIC_results(j, 1) = 2*k_gauss + 2*nll_gauss;
28     BIC_results(j, 1) = k_gauss*log(n) + 2*nll_gauss;
29
30     % --- MoG K=2 ---
31     mu_start = [quantile(x, 0.25), quantile(x, 0.75)];
32     sig_start = [std(x)*0.8, std(x)*1.2];
33     lam_start = [0.5, 0.5];
34     [p_mog2, nll_mog2] = fit_mog(x, mu_start, sig_start, lam_start
        );
35     params_other{j, 2} = p_mog2;
36     LL_results(j, 2) = -nll_mog2;
37     k_mog2 = 2*2 + (2-1);
38     AIC_results(j, 2) = 2*k_mog2 + 2*nll_mog2;
39     BIC_results(j, 2) = k_mog2*log(n) + 2*nll_mog2;
40
41     % --- MoG K=3 ---
42     mu_start = [quantile(x, 0.2), median(x), quantile(x, 0.8)];
43     sig_start = [std(x), std(x)*0.5, std(x)];
44     lam_start = [0.3, 0.4, 0.3];
45     [p_mog3, nll_mog3] = fit_mog(x, mu_start, sig_start, lam_start
        );
46     params_other{j, 3} = p_mog3;
47     LL_results(j, 3) = -nll_mog3;
48     k_mog3 = 3*2 + (3-1);
49     AIC_results(j, 3) = 2*k_mog3 + 2*nll_mog3;
50     BIC_results(j, 3) = k_mog3*log(n) + 2*nll_mog3;
51
52     % --- NCT ---

```

```

53     start_nct = [mean(x), std(x), 5, 0];
54     [p_nct, nll_nct] = fit_nct(x, start_nct);
55     params_other{j, 4} = p_nct;
56     LL_results(j, 4) = -nll_nct;
57     k_nct = 4;
58     AIC_results(j, 4) = 2*k_nct + 2*nll_nct;
59     BIC_results(j, 4) = k_nct*log(n) + 2*nll_nct;
60 end
61
62 save('fits_other.mat', 'params_other', 'LL_results', 'AIC_results'
    , 'BIC_results');
63
64 % 3. Weighted Chi-Square Estimation (K=2, 3, 4, 5)
65
66 K_vals = [2, 3, 4, 5];
67
68 for j = 1:nStocks
69     x = x_pct(:, j);
70     x = x(isfinite(x));
71
72     for K = K_vals positive
73         start_wchi = [min(x), ones(1, K)];
74
75         try
76             pars = fit_wchi_mle_one(x, K, start_wchi);
77
78             mu_hat = pars(1); a_hat = pars(2:end);
79             f_val = pdf_wghtd_sum(x - mu_hat, K, a_hat);
80             f_val = max(f_val, realmin);
81             LL = sum(log(f_val));
82             p_cnt = K + 1;
83             AIC = 2*p_cnt - 2*LL;
84             BIC = p_cnt*log(numel(x)) - 2*LL;
85
86             fname = sprintf('Estimated_parameters_WghtdChi2_K%d_%d
                .mat', K, j);
87             save(fname, 'pars', 'LL', 'AIC', 'BIC');
88         catch
89             fprintf('WChi Fit Failed: Stock %d, K=%d\n', j, K);
90         end
91     end
92 end

```

```

93
94 % 4. Plotting Stock 1 (Histogram + Densities)
95
96 x1 = x_pct(:, 1); x1 = x1(isfinite(x1));
97 x_grid = linspace(min(x1), max(x1), 1000);
98
99 wchi_pars = cell(1, 5);
100 for K = K_vals
101     fn = sprintf('Estimated_parameters_WghtdChi2_K%d_1.mat', K);
102     if isfile(fn), S=load(fn); wchi_pars{K} = S.pars; end
103 end
104
105 [kde_y, kde_x] = ksdensity(x1, x_grid);
106 pg = params_other{1, 1}; y_gauss = pdf_gauss(x_grid, pg(1), pg(2))
    ;
107 p2 = params_other{1, 2}; y_mog2 = pdf_mog(x_grid, p2.mu, p2.sigma
    , p2.lambda);
108 p3 = params_other{1, 3}; y_mog3 = pdf_mog(x_grid, p3.mu, p3.sigma
    , p3.lambda);
109 pn = params_other{1, 4}; y_nct = pdf_nct(x_grid, pn(1), pn(2),
    pn(3), pn(4));
110
111 figure; hold on;
112 histogram(x1, 100, 'Normalization', 'pdf', 'EdgeColor', 'none', '
    FaceColor', [0.8 0.8 0.8]);
113 plot(kde_x, kde_y, 'k--', 'LineWidth', 2);
114 plot(x_grid, y_gauss, 'LineWidth', 1.5);
115 plot(x_grid, y_mog3, 'LineWidth', 1.5);
116 plot(x_grid, y_nct, 'LineWidth', 1.5);
117
118 for K=K_vals
119     if ~isempty(wchi_pars{K})
120         p = wchi_pars{K};
121         y_w = pdf_wghtd_sum(x_grid - p(1), K, p(2:end));
122         plot(x_grid, y_w, 'LineWidth', 1.5);
123     end
124 end
125 title('Stock 1 In-Sample Fit');
126 hold off;
127
128 % 5. Value-at-Risk (VaR) Calculation
129

```

```

130 alpha = 0.01;
131 VaR_Table = NaN(nStocks, 8);
132
133 for j = 1:nStocks
134     x = x_pct(:, j); x = x(isfinite(x));
135     if numel(x) < 50, continue; end
136
137     x_min = quantile(x, 0.0001); x_max = quantile(x, 0.5);
138
139     % Empirical
140     [~, VaR_Table(j,1)] = var_empirical(x, alpha);
141
142     % Gauss
143     pg = params_other{j, 1};
144     [~, VaR_Table(j,2)] = var_gauss(pg(1), pg(2), alpha);
145
146     % MoG2
147     p2 = params_other{j, 2};
148     [~, VaR_Table(j,3)] = var_mog_grid(p2.mu, p2.sigma, p2.lambda,
        alpha, x_min, x_max);
149
150     % MoG3
151     p3 = params_other{j, 3};
152     [~, VaR_Table(j,4)] = var_mog_grid(p3.mu, p3.sigma, p3.lambda,
        alpha, x_min, x_max);
153
154     % NCT
155     pn = params_other{j, 4};
156     [~, VaR_Table(j,5)] = var_nct(pn(1), pn(2), pn(3), pn(4),
        alpha);
157
158     % WChi (K=2, 3, 5)
159     cols = [6, 7, 8];
160     ks    = [2, 3, 5];
161     for k_idx = 1:3
162         K = ks(k_idx);
163         fname = sprintf('Estimated_parameters_WghtdChi2_K%d_%d.mat', K, j);
164         if isfile(fname)
165             S = load(fname);
166             if isfield(S, 'pars')
167                 p = S.pars;

```

```

168             [~, VaR_Table(j, cols(k_idx))] = var_wchi_grid(p
169                 (1), p(2:end), alpha, x_min, x_max);
170         end
171     end
172 end
173
174 disp(array2table(VaR_Table, 'VariableNames', {'Emp', 'Gauss', 'MoG2',
    , 'MoG3', 'NCT', 'WChi2', 'WChi3', 'WChi5'}));

```

Listing 17: Main Script